

Chapter 4: Harmful Content Detection

Based on my study of *Machine Learning System Design Interview* (Aminian & Xu, 2023). The explanations, trade-off analysis and diagrams are my own. Notes marked **Beyond the book** are material I added.

The design in one paragraph. A post (text, images, video, links, context) goes through a cascade. First, cheap matching against known violating media. Then a **multimodal, multi-task model**: modality encoders → early fusion → shared layers → one head per harm category (violence, nudity, hate, self-harm, ...). The calibrated category scores feed a **policy layer** that combines degree and severity into an action: keep, demote, send to human review, or remove. It trains on large volumes of noisy user reports, is evaluated and calibrated on a smaller trusted human-labeled set, and goes to production shadow-first.

Chapter design map



Figure 1: Design map

1. Problem definition

- **Input:** a newly created or edited post, with text, images, video, links, post context, and selected author and engagement features.
- **Model output:** a calibrated probability per harm category.
- **Service action:** keep / demote (limit distribution) / human review / remove (and possibly record a violation). Also structured **reason codes** for a separate service that writes the user-facing explanation.

post → [p_violence, p_nudity, p_hate, p_self-harm, ...] → severity-aware policy → action + reason

Related but separate systems:

- **Bad-actor detection** aggregates an account's posts, reports, violations and behavior over time into an actor-level risk score. It uses different features, labels and actions (e.g. account restrictions).
- **Misinformation** follows the same design process, but its labels depend on fact-checking and changing context. Out of scope here.

From multimodal post to moderation action

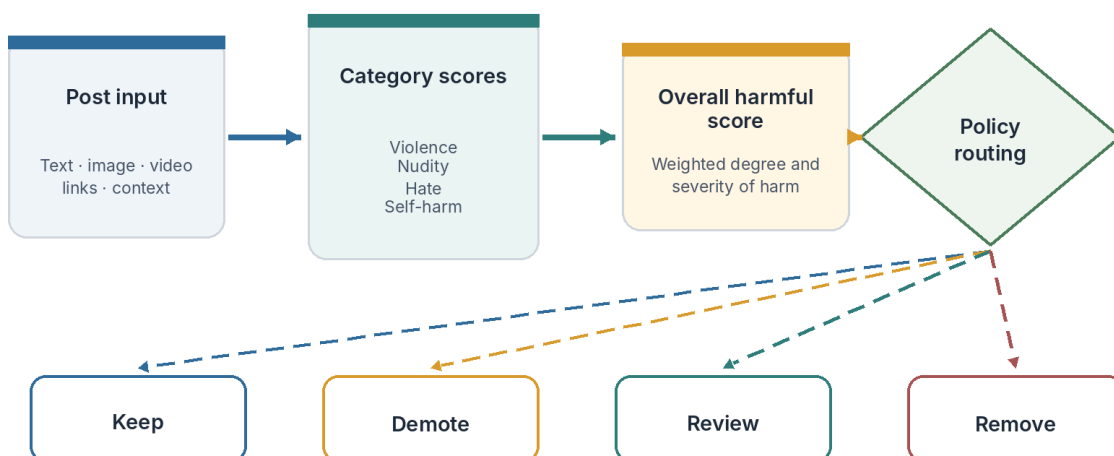


Figure 2: From post to action

2. Requirements

Functional: many languages; text, image and video, alone or combined; category scores and one action; reason codes.

Latency depends on severity:

Tier	Categories (example)	Timing
real-time	graphic violence, credible threats, child-safety violations, self-harm with intent	before or within seconds of publication
near-real-time	nudity, hate speech	minutes; can be demoted while pending
asynchronous	lower-severity policy violations	batch re-scoring, e.g. as comments and reports accumulate

This leads to a **cascade**:

```

every post → known-media hash match → cheap per-modality classifiers
                                         | (clearly safe → keep)
                                         ↓
                                full multimodal multi-task model
                                         | (uncertain → human review)
                                         ↓
                                policy action
  
```

Note. > **Beyond the book: hash matching first.** Much harmful media is re-uploaded. Perceptual hashes (e.g. PhotoDNA, or Meta’s open-source PDQ for images and TMK+PDQF for video) match near-duplicates of already-confirmed violating

content at almost no cost and very high precision. They're the right first stage. The ML model is for *new* content.

3. Labels and training data

Label strategy: scale and trust play different roles

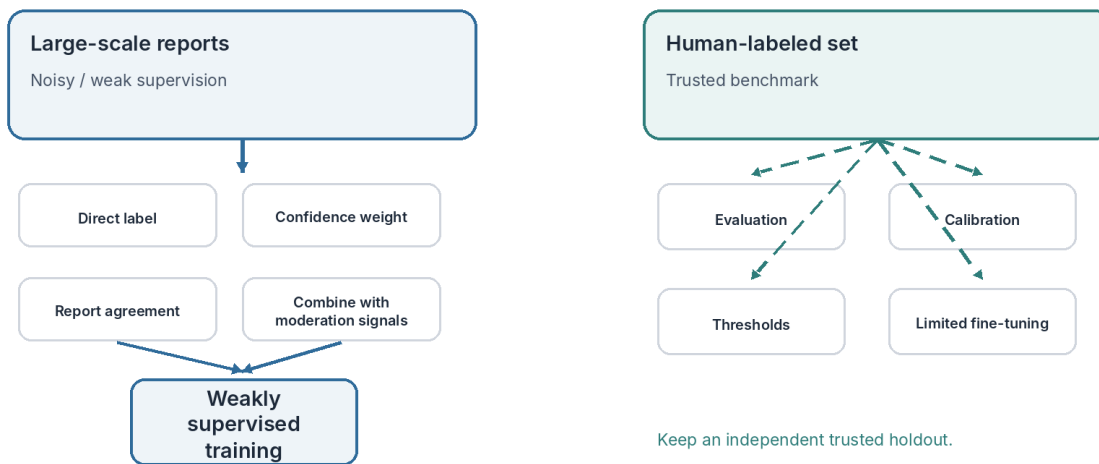


Figure 3: Label pipeline

Two label sources, with different roles:

Source	Scale	Quality	Use
user reports	millions	noisy: mass reporting, disagreements, wrong category	weakly supervised training
trusted human labels (policy-trained reviewers)	thousands to tens of thousands	high	evaluation, calibration, threshold setting; limited fine-tuning

How I'd turn reports into labels:

- a single report is weak evidence. Weight it by the reporter's historical accuracy;
- several independent reports raise confidence, but deduplicate coordinated reporting;
- if a moderator reviewed the report, use the moderator's decision as a strong label, including "not violating" as a strong negative;
- unreported posts are *mostly* safe but not labeled negatives. Sample them into training with lower weight.

Keep an independent trusted holdout. Never fine-tune on the evaluation set. I'd reserve about half of the human-labeled data, stratified by category and language, purely for evaluation and calibration.

4. Product metrics

Metric	Definition	Measures
prevalence (primary)	harmful impressions / total impressions, estimated by labeling a random sample of views	how much harm users actually see
content prevalence	harmful posts remaining visible / total posts	volume of harm on the platform
proactive rate	violating content removed before any user report / all violating content removed	how much the system finds on its own
valid-appeal (reversal) rate	enforcement decisions reversed on appeal / decisions appealed	wrongful enforcement
per-category and per-language breakdowns	each of the above, sliced	where the system is weak

Primary metric: impression-weighted prevalence, because a harmful post seen by a million people matters more than one seen by nobody. **Guardrails:** appeal reversal rate (over-enforcement), reviewer queue size and latency, creator retention.

5. Offline evaluation

- **PR-AUC and recall at a fixed precision** per category. Harmful content is rare (often < 1%), and precision-recall shows the positive-class trade-off directly.
- **ROC-AUC** is reported but not relied on. With millions of easy negatives, the false-positive *rate* stays tiny even when false positives outnumber true positives.
- **Calibration** per category, since thresholds and severity weights assume real probabilities.
- **Per-category thresholds.** A missed credible threat costs far more than a missed borderline insult, so the violence threshold for review is set lower. I'd set each threshold from the trusted set at a target precision for automatic removal (e.g. $\geq 95\%$) and a target recall for review.

6. Early vs late fusion

	Late fusion	Early fusion
structure	separate text / image / video classifiers, scores combined at the end	modality embeddings concatenated (or cross-attended) and fed to one shared model
strengths	modular, debuggable, each modality deployable on its own, missing modalities easy	learns cross-modal meaning
weaknesses	misses meaning that only exists in combination	harder to train; missing modalities need care; more compute

A harmless photo plus a harmless caption can be hateful together (the classic “hateful meme” problem), and late fusion can’t see that. So the **preferred design is early fusion, with late fusion as the baseline**. Adopt early fusion if it improves PR-AUC on a trusted set of multimodal posts (not just overall) by a meaningful margin.

Multimodal fusion: modular baseline vs cross-modal learning

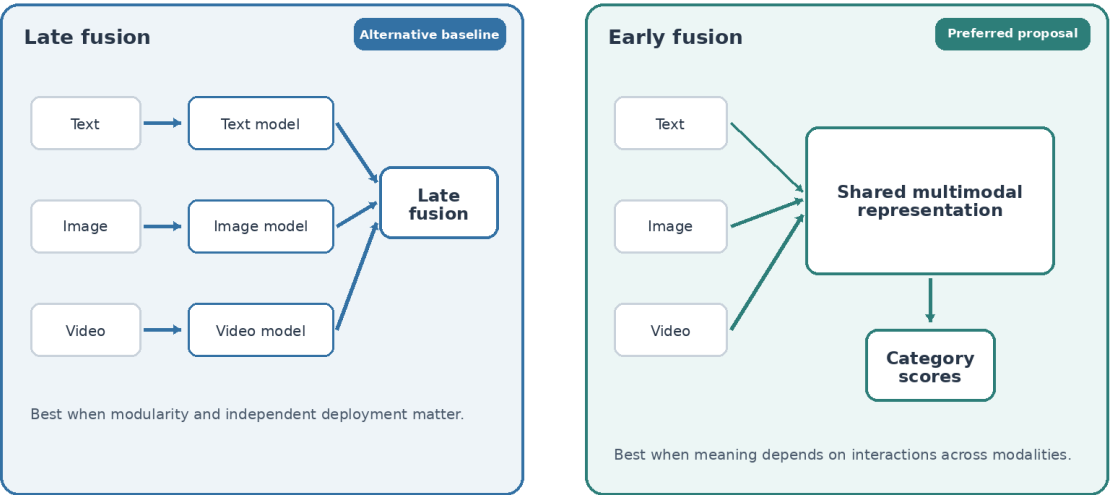


Figure 4: Fusion comparison

Classifier alternatives: from simple action to granular tasks

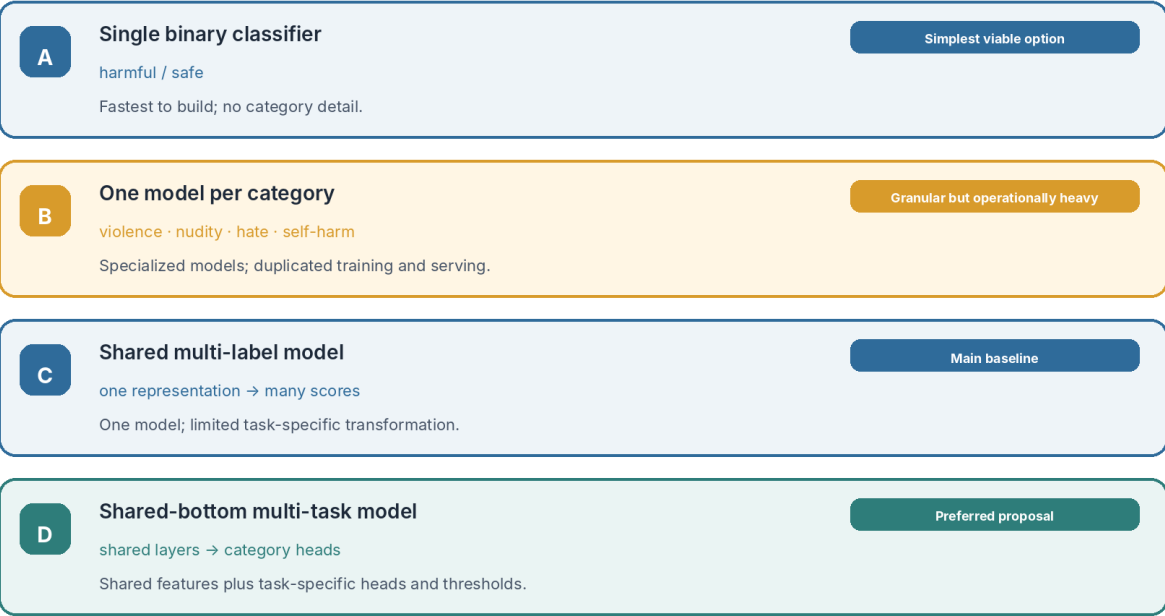


Figure 5: Model alternatives

7. Classifier structure

Option	Description	Pro	Con
1. single binary	harmful vs safe	simplest	no category, so no category thresholds or reason codes
2. one model per category	separate violence, nudity, ... models	specialized	N models to train, serve and keep consistent
3. shared multi-label	one network, one sigmoid output per category	one model, independent scores	every category shares the same final layer
4. shared-bottom multi-task	shared layers + a small head per category	shared features <i>and</i> per-category transformations, thresholds and loss weights	more tuning; task gradients can conflict

Choice: option 4, with option 3 as the baseline. Give a category its own deeper head only if it underperforms when it shares, e.g. self-harm, whose language differs a lot from violence.

8. Modality encoders

Pretrained encoders, fine-tuned or frozen with trainable projection layers:

Modality	Options	Why
text	multilingual transformer: XLM-R, LaBSE, or a distilled multilingual model for latency	one model across ~100 languages; DistilBERT and Sentence-BERT are English-centric unless you use their multilingual variants
image	CLIP ViT image encoder; a ResNet/ViT pretrained with a self-supervised method like SimCLR	CLIP's image-text alignment helps text-in-image and meme understanding
video	sample frames → image encoder → temporal pooling or attention; or a video transformer (VideoMAE, X-CLIP), possibly pretrained with a self-supervised method like VideoMoCo	cost grows with length, so sample frames and add audio transcripts as text

(SimCLR and VideoMoCo are *training methods*, not architectures: they produce encoders.)

9. Missing modalities

Posts can be text-only, image-only, text + video, and so on. The model must tell “no image” apart from “a harmless image”.

Method	Pro	Con
zero vector	trivial	can look like a real low-information embedding
learned “missing” embedding per modality + a presence flag	the model knows what’s absent	one extra parameter vector per modality
modality-specific handling (e.g. empty-string text encoding)	natural for some encoders	inconsistent across modalities

Choice: a learned missing embedding plus a presence flag for each modality, trained with **modality dropout** (randomly hiding a present modality during training) so the model stays robust when one is missing.

10. Additional features

Engagement (impressions, likes, shares, reports so far): useful for re-scoring, but zero at upload time. Only use values available at the moment of prediction.

Comments: they arrive after posting, so they don’t help the first decision. I’d use them for **re-scoring**: mean-pool the embeddings of up to N recent comments (the simple default), or use attention pooling or max-risk pooling if a few alarming comments matter more than the average.

Author history (prior violations, report rate, account age, follower counts): strongly predictive, but it risks penalizing people for their past rather than this post, and it can encode bias. My choice: keep the harm score **content-only**, and use author history for **routing and prioritization** (e.g. faster review for repeat violators) and in the separate actor-level system. Only history before the current post may be used.

Context: posting time, device, country, surface (public vs group).

11. From category scores to an action

Several categories can be present at different levels. Options for combining them:

Method	Pro	Con
weighted sum $\sum_c s_c p_c$	interpretable	weights need policy input
max category	sensitive to one severe category	ignores several moderate ones
learned aggregator	captures interactions	opaque; needs overall labels
hybrid	policy control + flexibility	more to maintain

My choice (hybrid): per-category calibrated scores are checked against per-category thresholds **first**, so severe categories can trigger removal on their own. Otherwise, a severity-weighted risk $R = \max_c (s_c p_c)$, with severities s_c set by policy, plus a small bonus when several categories are moderately high, drives the demote / review decision.

12. Training the multi-task model

Each head has a binary cross-entropy loss, and the shared layers receive the sum. Two different balancing problems are easy to confuse:

Training: balance tasks, modalities, and rare classes

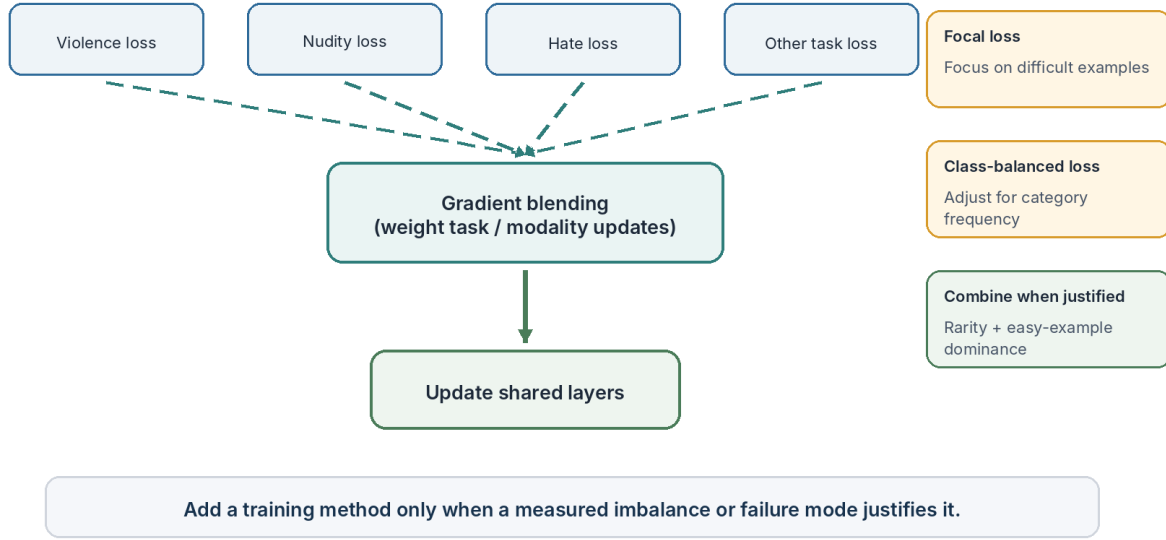


Figure 6: Training flow

- **Balancing tasks:** some category losses are larger or noisier and dominate the shared gradients. **GradNorm** (Chen et al., 2018) learns loss weights that equalize per-task gradient norms. Uncertainty weighting (Kendall et al., 2018) is an alternative.
- **Balancing modalities:** in multimodal training, one modality (often text) is learned fast and overfits while others are under-trained. **Gradient Blending** (Wang, Tran & Feiszli, 2020) weights each modality’s loss by its estimated overfitting-to-generalization ratio.

I’d first measure which imbalance exists (per-task gradient norms, per-modality validation curves), then apply the matching fix. Evidence that it helped: rare-category PR-AUC improves without a drop in the frequent categories.

13. Class imbalance

Harm is rare overall, and some categories are far rarer than others.

Focal loss (Lin et al., 2017) down-weights easy examples:

$$\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t, \quad \gamma \approx 2.$$

An easy negative with $p_t = 0.99$ has its loss scaled by $0.01^2 = 10^{-4}$. Risk: noisy reports look like “hard” examples and get *more* weight.

Class-balanced loss (Cui et al., 2019) weights class c by the inverse of its *effective number* of samples, $E_{n_c} = (1 - \beta^{n_c}) / (1 - \beta)$. With $\beta = 0.9999$, a category with 100 examples gets about $100\times$ the weight of one with 100,000, instead of the $1,000\times$ that plain inverse frequency would give. That tames overweighting of noisy rare classes.

They address different things (difficulty vs rarity) and can be multiplied, but add each only when its failure mode is measured: easy negatives dominating the loss, or a rare category’s recall collapsing.

14. Long inputs and attention cost

Standard self-attention costs $O(n^2)$ in sequence length. Long posts, transcripts and many video frames get expensive. Options, in order:

1. **reduce n** : truncate, sample frames, pool patches (often enough);
2. **efficient attention**: linear or kernelized attention, low-rank (Linformer), or sparse/local windows. Each is cheaper and may lose some long-range interaction quality.

Switch when p99 latency or GPU memory is exceeded at the real length distribution, and accept the change only if category PR-AUC drops by less than a set tolerance.

15. Serving and moderation policy

Serving and rollout: risk-sensitive paths

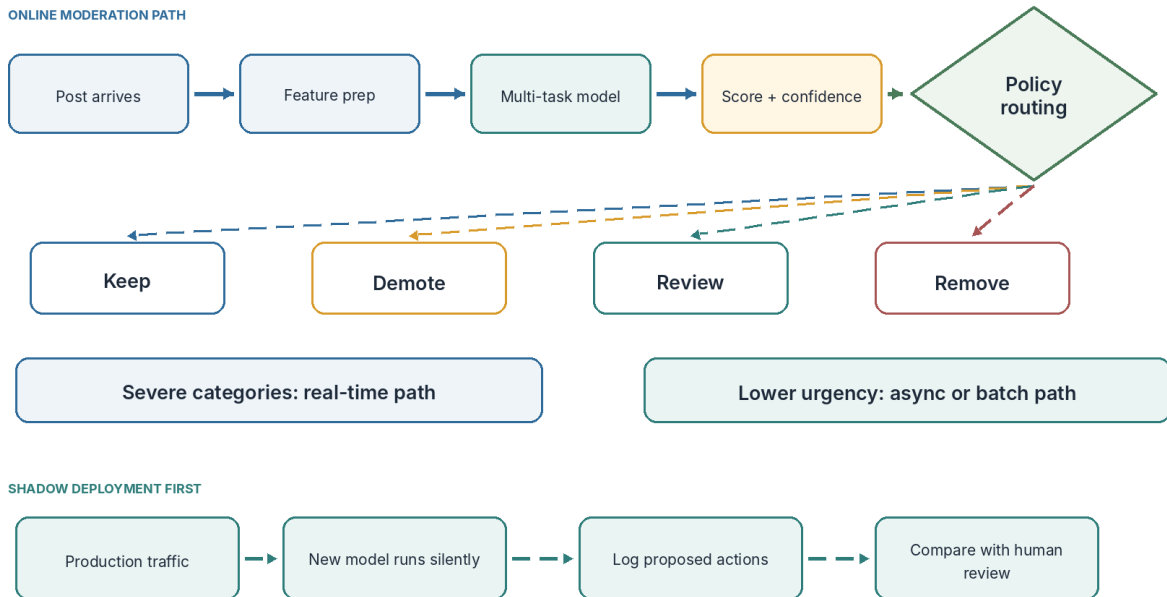


Figure 7: Serving and shadow deployment

A starting policy (thresholds on calibrated scores, set per category from the trusted set):

Condition	Action
any category $\geq$ its removal threshold (precision $\geq$ 95%)	remove; record a violation; send reason codes
severity-weighted risk in the review band, or model uncertainty high	human review queue, prioritized by severity $\times$ reach
risk in the demote band	limit distribution while pending
otherwise	keep

Severe categories get lower review thresholds, and some (credible threats) always go to review. Queue capacity is part of the design: if the review band sends more items than reviewers can handle, the threshold has to move.

16. Deployment: shadow first

A/B testing a moderation model means knowingly exposing some users to more harm, or wrongly removing their posts. So the first production step is **shadow deployment**:

production traffic → new model scores silently → proposed actions logged
→ compared with the current system and with human review of disagreements
→ limited enforcement (one category, one region) → full rollout

Shadow can validate latency, reliability, score distributions, agreement with production, the disagreement cases and per-category precision on reviewed samples. It **can't** measure how users respond to real enforcement. That comes with limited enforcement, and A/B tests are reserved for lower-risk changes (e.g. demotion thresholds).

Exit criteria I'd set: p99 latency within budget for two weeks; reviewed precision at the removal threshold $\geq$ production in every category; no category or language with recall below production; disagreement cases reviewed and explained.

17. End-to-end design

One reasonable design under the current assumptions

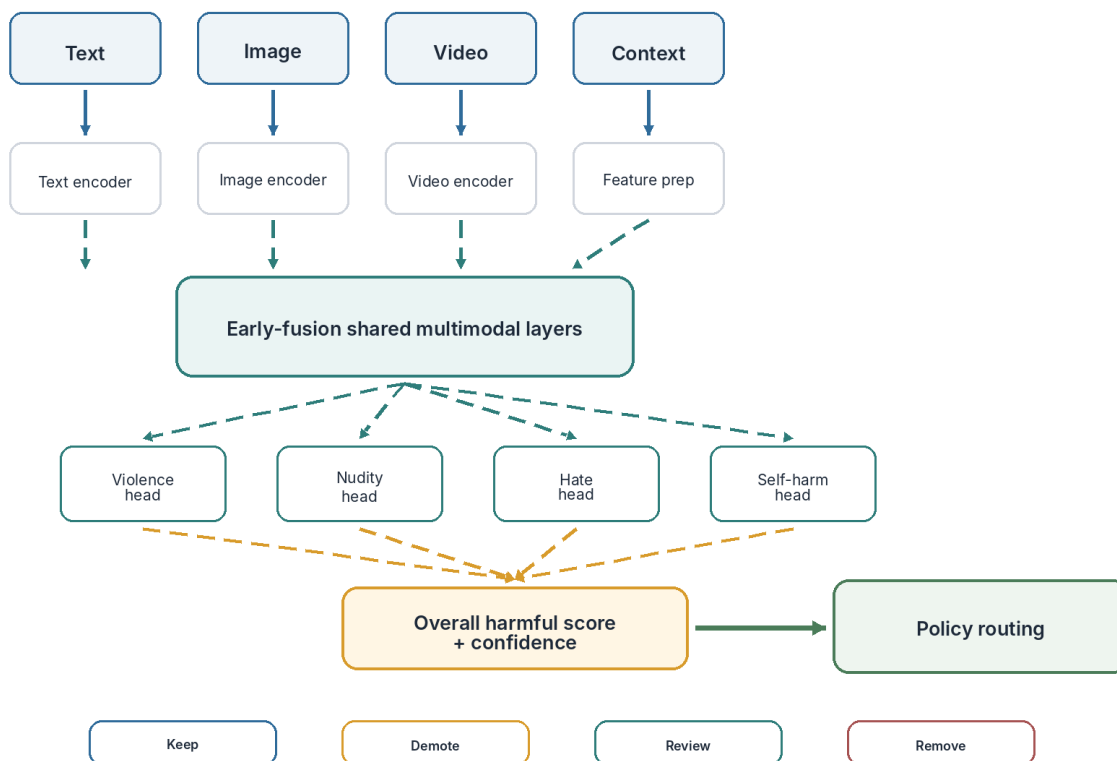


Figure 8: Reference architecture

post → hash match → (match) → remove
→ text / image / video encoders (+ missing embeddings) → early fusion → shared layers
→ category heads → calibrated scores → severity-aware policy
→ keep / demote / review / remove (+ reason codes → explanation service)
account history over time → actor-level risk (separate system)

18. My decisions

Decision	Choice	Would change if...
main categories	violence, nudity, hate, self-harm, plus others by policy	policy adds or merges categories
real-time categories	violence, credible threats, child safety, self-harm with intent	latency cost is prohibitive → a cheaper real-time model for these only
user reports	reliability-weighted weak labels; moderator decisions as strong labels	a noise-robust loss beats weighting on the trusted set
human labels	half held out for evaluation/calibration, half for fine-tuning	-
primary product metric	impression-weighted prevalence	-
primary offline metric	per-category PR-AUC and recall at the removal-precision target	-
fusion	early fusion; late fusion as baseline	early fusion gains < ~1 pt PR-AUC on multimodal posts
architecture	shared-bottom multi-task; multi-label baseline	the baseline is within noise on every category
harm score	per-category thresholds first, then severity-weighted max	a learned aggregator beats it on overall labels and passes policy review
task / modality balancing	measure first; GradNorm for tasks, Gradient Blending for modalities	no measurable imbalance → plain summed loss
imbalance	class-balanced weights; add focal loss if easy negatives dominate	-
missing modality	learned missing embedding + presence flag + modality dropout	-
comments	re-scoring only, mean-pooled	a few alarming comments are missed → max-risk pooling
author history	routing and priority only, not in the harm score	policy decides author history is a legitimate input
rollout	shadow → limited enforcement → full	-

19. Main lessons

1. Multi-label inside, one action outside. Category scores enable thresholds, severity and reason codes.
2. Reports give scale; human labels give truth. Keep a trusted, independent evaluation set.
3. Early fusion is justified when meaning lives in the *combination* of modalities.
4. Balance tasks and modalities only after measuring which one is out of balance.
5. Focal loss addresses difficulty, class-balanced loss addresses rarity. They're different tools.
6. Represent missing modalities explicitly and train for their absence.
7. Moderation is a spectrum (keep, demote, review, remove), and reviewer capacity is part of the system.

8. Go shadow-first. Enforcement mistakes are costly in both directions.
